Shared Laboratory Equipment Booking System

A C++ console application that records who has which lab instrument, when it is due, and what a late return costs.

Team Ansh Arjun Lakshit Priyanshu **Repo** github.com/vee1e/lab-booking-system

1. Problem

The lab shares its microscopes, centrifuges and spectrometers between students and staff. Nothing records who holds what, so two people can claim the same instrument, and a late return leaves no trace. We built a menu-driven program that keeps that record in memory: it tracks each item's state, each user's quota and loan period, and every borrow and return.

2. How it works

STEP 01
Sign in
Pick a user. The role sets the item limit, loan length and fine rate.

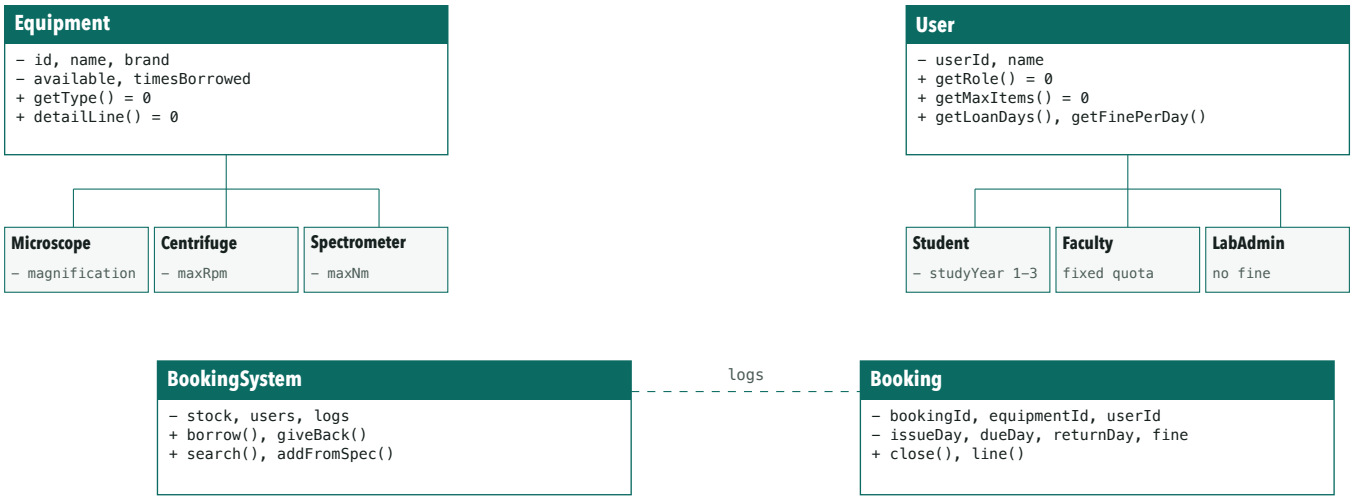
STEP 02
See the shelf
Menu 2 lists every instrument as FREE or TAKEN, with its type and spec.

STEP 03
Borrow
Checks availability and quota, then writes a booking with a due day.

STEP 04
Return
Closes the booking and charges the fine if the return is past due.

Other menu actions: booking log, search by word or id, user list, admin add, admin remove, advance day, exit.

3. Class design



Solid lines show inheritance. The dashed line shows that BookingSystem stores Booking records by value, while items and users are owned through pointers.

4. Role rules

ROLE	MAX ITEMS	LOAN	FINE PER LATE DAY	SOURCE
Student, year 1	2	7 days	10	`User.h`
Student, year 2	3	7 days	10	`User.h`
Student, year 3	4	7 days	10	`User.h`
Faculty	5	14 days	5	`User.h`
Lab Admin	10	30 days	0	`User.h`

The fine rate and loan length come from the signed-in user's role. Booking due day = issue day + loan days.

5. Three states on screen

FREE
On the shelf
The item is free. Anyone under their item limit can borrow it.

TAKEN
Already out
A second borrow is refused, and the message names the holder.

LATE
Past the due day
The return is accepted, and the days late and the amount owed appear.

6. OOP features in the code

FEATURE	WHERE IT IS USED
Encapsulation	All state is private. `available` changes only through `markBorrowed()` and `markReturned()`; a booking closes only through `close()`. Outside code reads state through const getters.
Inheritance	`Equipment` has three children (`Microscope`, `Centrifuge`, `Spectrometer`); `User` has three (`Student`, `Faculty`, `LabAdmin`). Each child adds one field or one rule set.
Abstraction	`getType()` and `detailLine()` are pure virtual, and so are the four `User` rule methods. No `Equipment` or `User` object can exist on its own.
Polymorphism	The system holds `vector<Equipment*>` and `vector<User*>`. List, search and quota checks call the child version through the base pointer, with no type checks.
Overloading	Two constructors for each instrument type and for `Student` (full spec or default), and two `search` functions, one taking a `string` and one taking an `int`.
Composition	`BookingSystem` owns the item and user lists and deletes them in its destructor. `Booking` stores ids instead of pointers, so an old record cannot dangle after an item is removed.

Also used: static id counters shared by each class, default arguments, friend `operator<<`, and two plain structs, `Result` (ok flag plus message) and `GearSpec` (one creation pack).

7. Borrow rules and refusals

Borrowing is refused when the item is already out (the message names the holder) or the user is at their role's item limit (the message gives the count and the limit). Removing an item is refused while it is still out.
Borrow and return also require a signed-in user. Borrow, return, add and remove each return a `Result` with a full sentence, so the screen never shows a bare error code. Non-numeric menu input is rejected and re-asked instead of leaving `cin` in a failed state.

Rules live in one place, `BookingSystem::borrow` and `giveBack`. The menu only reads input and prints the message, so a rule change does not touch the interface.
The log keeps closed records too, so the history stays readable after an item is back on the shelf.

```
B5001 gear=101 user=1 issue=1 due=8 back=9 fine=10
```

8. Sample run

```
# borrow the microscope, request it again, then return on day 9)
> 3   101
OK: Priyanshu took 'Olympus CX23' [Microscope] on day 1. Due day 8 (loan 7 days for Student).)
> 3   101
FAIL: 'Olympus CX23' is TAKEN by user 1. Pick another id from menu 2.)
> 4   101
OK but LATE: day 9 is past due 8. Fine=10 (1 day x 10/day).)
```

Output from the built binary, trimmed to the lines that matter. The full run also covers quota refusal, a second user, search by word and id, and admin add and remove.

9. Limits and next steps

CURRENT LIMITS

- Nothing is saved. Closing the program resets the register.
- Dates are a manual day counter, not real dates.
- One terminal session at a time, with no concurrent users.
- Menus 9 and 10 do not check for the Lab Admin role.
- Checks are manual. There are no automated tests.

NEXT STEPS

- Save and load the register from a file.
- Use real dates for issue and due days.
- Restrict add and remove to the admin role.
- Add unit tests for the quota, availability and fine rules.

10. Build

Build with `g++ -std=c++17 -Wall main.cpp -o labbook`, then run `./labbook`. C++17 and the standard library only, with no database and no third-party dependency. The same menu and rules are also ported to the browser at lab-booking-system-mu.vercel.app/#shell.

The build is clean under `-Wall`. A scripted run covered login, a borrow, duplicate and quota refusals, a late fine, both searches, and admin add and remove.